

Алгоритмы генерации

Генерация пермутаций л.ч.

1, 2, 3, 4, ..., n

1 2 3 4 5 - - (n-1) n

1 2 3

(n-2) n (n-1)

1 2 3

(n-1) (n-2) n

1 2 3

(n-1) n (n-2)

1 2 3

n (n-2) (n-1)

...

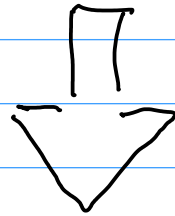
Ogólne zasady generowania
nast. lex permutacji najmniejszej w sufiksach
 $> a_{n-k}$

$a_1 a_2 a_3 \dots$

$a_{n-k} a_{n-k+1} \dots a_{n-1} a_n$

najdłuższy należący
sufiks

przed i następnik



$a_1 a_2 \dots a_{n-k-1} a_{n-k} \dots a_{n-1} a_n$

zamiast a_{n-k}



Czas tworzenia następnika $\leq ck$

Najdłuższy malejący sufix permutacji
 ma dł $\geq k$ dla $\frac{n!}{k!}$ permutacji
 o dł k



może być w
 jednym z $k!$ przedziałów
 (tylko jeden jest malejący)

P_k - liczba permutacji
 o najdł sufixie k -el

Powyższe sko możemy zapisać nast:

$$P_k + P_{k+1} + P_{k+2} + \dots = \frac{n!}{k!}$$

čas generovania vsy stihich permut
lex je $\leq$ najvyzej

$$P_1 \cdot c \cdot 1 + P_2 \cdot c \cdot 2 + P_3 \cdot c \cdot 3 + \dots + P_n \cdot c \cdot n$$

$$= c \left((P_1 + P_2 + \dots) + (P_2 + P_3 + \dots) + (P_3 + P_4 + \dots) + \dots \right)$$

$$= c \left(\frac{n!}{1!} + \frac{n!}{2!} + \frac{n!}{3!} + \dots \right) = c n! \left(\frac{1}{1!} + \frac{1}{2!} + \dots \right)$$

$$\leq c n! (e - 1)$$

vyli zamorty rovnay cas generovania
poj permutacij $\leq c(e - 1)$

Algorytm generujący minimalny Δ zmian

- każde kolejne dwa wygenerowane obiekty różnią się minimalną zmianą.

Motywacja

- w pierwszym zastosowaniu potrzebujemy, żeby kolejne wygenerowane obiekty różniły się minimalną zmianą
- niewielkie różnice między kolejnymi obiektami ułatwiają sprawdzenie pierwszej cięsnosci
- niewielkie zmiany mogą się przekładać na niższy czas działania programu generującego

Algoritm generovania v systēch
 čísel 2er i jedynok alT n

minimálne zmena - zmena
 jednej pozície.

00
 01
 11
 10

000
 001
 011
 010
 110
 111
 101
 100

odvrácanie

0000
 0001
 0011
 0010
 0110
 0111
 0101
 0100
 1100
 1101
 1111
 1110
 1010
 1011
 1001
 1000

Alg generovania
 kodu Gray'a

Implementacja :
✓ bryła i jeśli $2^{p-1} \mid i \wedge 2^p \nmid i$
zrob zmianę na pozycji p

Pokazujemy teraz jak generować
permutacje ze pom alg
minimalnych zmian

Minimalne zmiany: sąsiednia transpozycja

Sposób generowania jest wzięty z
Trotterem, Johnsonem, Steinhausem

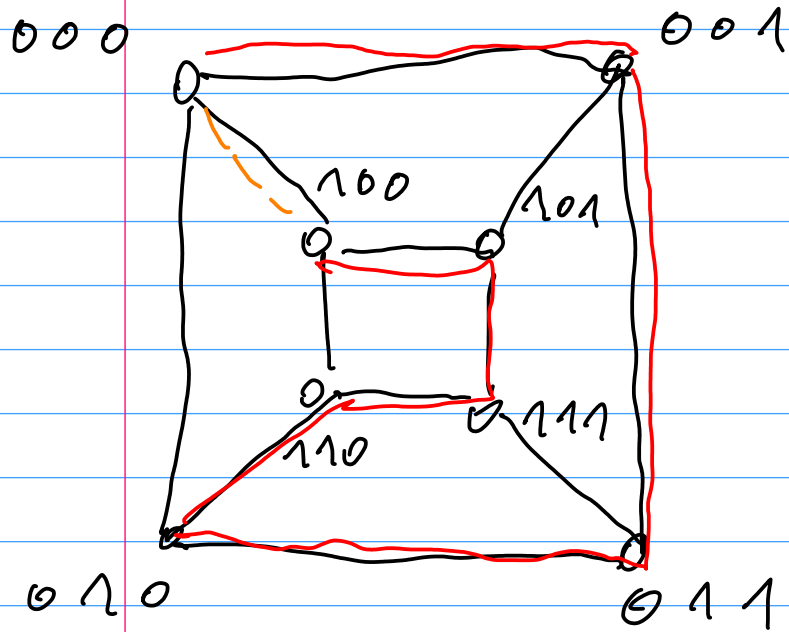
1 2
2 1

1 2 3 ← 1
1 3 2 ← 2
2 1 2 ← 3
3 2 1
2 3 1
2 1 3

1 2 3 4
1 2 4 3
1 4 2 3
4 1 2 3 ← 1
4 1 3 2 ← 2
1 4 3 2
1 3 4 2
1 3 2 4 ← 2
3 1 2 4 ← 3
3 1 4 2
3 4 1 2
4 3 1 2

4 3 2 1
3 4 2 1
3 2 4 1
3 2 1 4
2 3 1 4
2 3 4 1
2 4 3 1
4 2 3 1
4 2 1 3
2 4 1 3
2 1 4 3
2 1 3 4

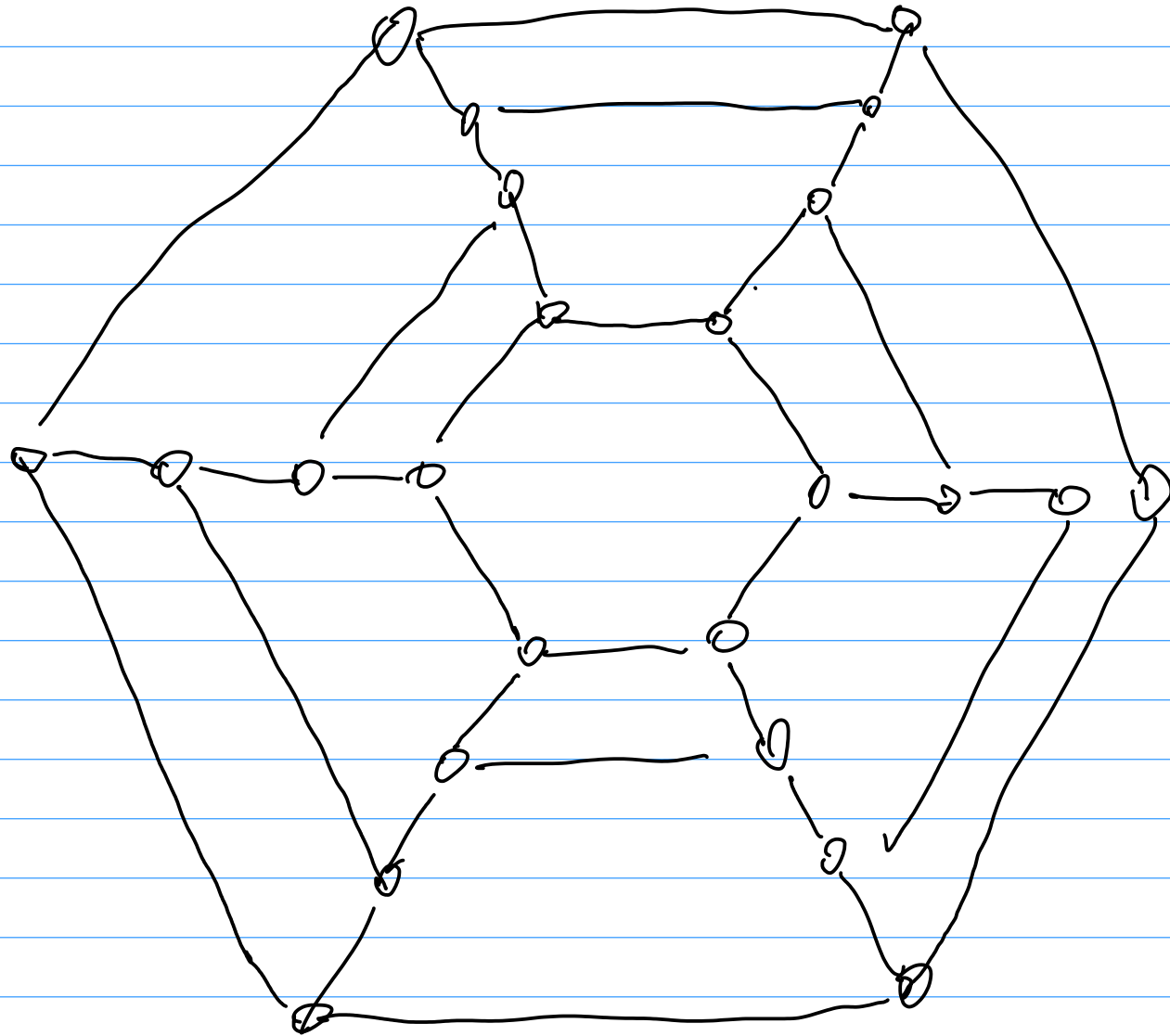
Graf minimalnych zmiar



alg min zmiar

≡ ścieżka

Hamiltona



Graf dla permutacji